

INSTITUTO FEDERAL DE EDUCACAO, CIENCIA E TECNOLOGIA DO TOCANTINS
CAMPUS PARAISO DO TOCANTINS
CURSO DE BACHARELADO EM SISTEMAS DE INFORMACAO

DOCUMENTO DE ESPECIFICACAO:
T.O.D.A.N (TRUTH OR DARE ASYNC NETWORK)

Disciplina: Linguagens e Tecnicas de Programacao IV

Professor: Marcos Raimundo Mendes Ramos

Equipe de Desenvolvimento:

- Pedro Roberto Ribeiro Bandeira Labre

Paraiso do Tocantins - 2026

1. HISTORICO DE ALTERACOES

Data	Versao	Descricao da Alteracao	Autor
24/03/2026	1.0	Inicio da fase de prototipacao visual (light mode) e definicao inicial do escopo do aplicativo.	Pedro Roberto Ribeiro Bandeira Labre
25/03/2026	1.1	Evolucao dos prototipos light mode e organizacao dos fluxos no README para documentacao visual.	Pedro Roberto Ribeiro Bandeira Labre
26/03/2026	1.2	Conclusao da rodada de prototipos dark mode (busca, perfil, clubes, grupos, respostas e configuracoes).	Pedro Roberto Ribeiro Bandeira Labre
28/03/2026	1.3	Inicializacao tecnica do backend (Express, Prisma, PostgreSQL) e implementacao da autenticacao com testes automatizados.	Pedro Roberto Ribeiro Bandeira Labre
28/03/2026	1.4	Inicializacao da base do app mobile com tela de login e implementacao da tela de cadastro integrada ao fluxo de autenticacao.	Pedro Roberto Ribeiro Bandeira Labre
29/03/2026	1.5	Integracao da autenticacao mobile com backend e consolidacao dos testes de frontend para login e cadastro.	Pedro Roberto Ribeiro Bandeira Labre
30/03/2026	1.6	Implementacao da tela de feed mobile e configuracao de integracao da API por variaveis de ambiente.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.7	Implementacao do backend do feed com testes e isolamento do banco de testes para execucoes automatizadas.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.8	Entrega de criacao de truths e dares no backend, com cobertura de testes e integracao com feed.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.9	Implementacao da tela create-challenge no mobile com consumo real de usuarios, truths e dares.	Pedro Roberto Ribeiro Bandeira Labre
04/04/2026	1.10	Aprimoramentos do feed: persistencia de usuario-alvo, configuracoes de dare e regras de permissao para delete.	Pedro Roberto Ribeiro Bandeira Labre
05/04/2026	1.11	Integracao de status real de dares e likes persistidos no feed, com melhorias de comportamento de interface.	Pedro Roberto Ribeiro Bandeira Labre
06/04/2026	1.12	Implementacao da tela de comentarios do feed no mobile, com estrutura de fluxo pronta para evolucao backend.	Pedro Roberto Ribeiro Bandeira

Data	Versao	Descricao da Alteracao	Autor
			Labre
07/04/2026	1.13	Implementacao da base de perfil, configuracoes e notificacoes no mobile com tema global compartilhado.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.14	Consolidacao das telas de busca, clubes e criacao de grupo; integracao de perfil real (username/bio) e logout funcional.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.15	Atualizacao da documentacao com analise tecnica detalhada de backend e mobile, incluindo escopo implementado e integracoes pendentes.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.16	Ajuste do build de producao do backend para viabilizar deploy online no Render com banco Supabase (commit 0e16e75).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.17	Correcao dos imports do Prisma Client no artefato compilado para estabilidade do backend online no Render (commit 17cbd76).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.18	Ajuste do prisma.config para migrations com DIRECT_URL do Supabase, garantindo fluxo de deploy (commit a32427f).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.19	Correcao do entypoint de start em producao para manter a API online no Render (commit 4a6ffb1).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.20	Consolidacao da publicacao do backend em nuvem: aplicacao no Render e banco PostgreSQL gerenciado no Supabase.	Pedro Roberto Ribeiro Bandeira Labre

2. DEFINICAO DO PROJETO

2.1 Tema

Rede social assincrona baseada em Verdade ou Desafio, com interacoes por prompts de verdade/desafio, feed social e dinamicas em grupos (clubs).

2.2 Objetivo do Aplicativo

O aplicativo busca oferecer uma experiencia social de entretenimento em que usuarios criam e respondem desafios de forma assincrona, sem depender de todos estarem conectados ao mesmo tempo. O foco e estimular engajamento por meio de perfis, feed, curtidas e participacao em grupos.

3. REQUISITOS DO SISTEMA

3.1 Requisitos Funcionais (RF)

Descrevem as acoes que o sistema deve ser capaz de executar.

- RF001: O sistema deve permitir cadastro de usuario por email e senha.
- RF002: O sistema deve permitir autenticacao de usuario por email e senha.

- RF003: O sistema deve emitir token JWT no login autenticado.
- RF004: O usuario autenticado deve conseguir consultar o proprio perfil.
- RF005: O usuario autenticado deve conseguir atualizar o proprio perfil.
- RF006: O sistema deve permitir listar usuarios para interacao.
- RF007: O sistema deve permitir criar prompts do tipo verdade para outro usuario.
- RF008: O sistema deve permitir criar prompts do tipo desafio para outro usuario.
- RF009: O sistema deve permitir excluir uma verdade quando houver permissao.
- RF010: O sistema deve permitir excluir um desafio quando houver permissao.
- RF011: O sistema deve disponibilizar feed autenticado com itens recentes.
- RF012: O sistema deve permitir curtir/descurtir truths.
- RF013: O sistema deve permitir curtir/descurtir dares.
- RF014: O sistema deve permitir curtir/descurtir clubs.
- RF015: O app mobile deve disponibilizar tela de login.
- RF016: O app mobile deve disponibilizar tela de cadastro.
- RF017: O app mobile deve disponibilizar tela de feed.
- RF018: O app mobile deve disponibilizar tela de comentarios do feed.
- RF019: O app mobile deve disponibilizar tela de criacao de desafio.
- RF020: O app mobile deve disponibilizar tela de busca.
- RF021: O app mobile deve disponibilizar tela de clubes.
- RF022: O app mobile deve disponibilizar tela de criacao de grupo.
- RF023: O app mobile deve disponibilizar tela de perfil.
- RF024: O app mobile deve disponibilizar tela de configuracoes.
- RF025: O app mobile deve persistir token de autenticacao localmente.
- RF026: O app mobile deve enviar token nas requisicoes protegidas.
- RF027: O app mobile deve listar usuarios no fluxo de criacao de desafio.
- RF028: O app mobile deve carregar dados do proprio perfil pela API.
- RF029: O app mobile deve atualizar dados do proprio perfil pela API.
- RF030: O app mobile deve realizar logout removendo token local.
- RF031: O app mobile deve redirecionar para login apos logout.
- RF032: A busca mobile deve manter estrutura pronta para integracao backend.
- RF033: O modulo de clubes mobile deve manter estrutura pronta para integracao backend.
- RF034: O modulo de criacao de grupo mobile deve manter estrutura pronta para integracao backend.
- RF035: O modulo de comentarios mobile deve manter estrutura pronta para integracao backend.

3.2 Requisitos Nao Funcionais (RNF)

Descrevem caracteristicas de qualidade, desempenho e restricoes.

- RNF001: A API deve utilizar comunicacao HTTP.
- RNF002: A API deve utilizar payload em JSON.
- RNF003: Rotas protegidas devem exigir autenticacao por JWT.
- RNF004: Senhas devem ser armazenadas com bcrypt.
- RNF005: O backend deve ser implementado em TypeScript.
- RNF006: O backend deve compilar via TypeScript compiler.
- RNF007: O backend deve possuir testes automatizados com Jest.

- RNF008: O mobile deve possuir testes automatizados com Jest.
- RNF009: A persistencia deve utilizar PostgreSQL.
- RNF010: O acesso ao banco deve utilizar Prisma ORM.
- RNF011: O codigo-fonte deve ser versionado com Git.
- RNF012: O repositorio deve ser mantido no GitHub.
- RNF013: O mobile deve utilizar Expo Router para navegacao.
- RNF014: O mobile deve manter stack centralizada de rotas.
- RNF015: O mobile deve suportar tema system.
- RNF016: O mobile deve suportar tema light.
- RNF017: O mobile deve suportar tema dark.
- RNF018: O token de autenticacao no mobile deve ser persistido com AsyncStorage.
- RNF019: O consumo de API no mobile deve ser centralizado em camada de servico.
- RNF020: O consumo de API no mobile deve aplicar tratamento padrao de erro.
- RNF021: A arquitetura mobile deve separar componentes reutilizaveis.
- RNF022: A arquitetura mobile deve separar hooks de estado e fluxo.
- RNF023: A arquitetura mobile deve separar constantes de tema e UI.
- RNF024: A arquitetura mobile deve separar tipos compartilhados.
- RNF025: O build de producao do backend deve executar prisma generate antes da compilacao TypeScript.
- RNF026: O backend em producao deve iniciar pelo entrypoint compilado dist/src/server.js.
- RNF027: As migrations do backend devem usar conexao direta via variavel DIRECT_URL.
- RNF028: O runtime do backend deve conectar no banco pela variavel DATABASE_URL.
- RNF029: O servidor backend deve fazer bind em 0.0.0.0 com porta via variavel PORT.
- RNF030: O backend deve estar publicado em ambiente de producao no Render.
- RNF031: O banco de producao deve utilizar PostgreSQL gerenciado no Supabase.
- RNF032: O deploy de producao deve aplicar migrations com prisma migrate deploy antes do start da API.
- RNF033: O backend deve exigir as variaveis DATABASE_URL, DIRECT_URL e JWT_SECRET no ambiente de producao.
- RNF034: O mobile deve exigir a variavel EXPO_PUBLIC_API_URL para consumo da API em ambiente publicado.

3.3 Modelagem

Modelagem de alto nivel das funcionalidades e telas do aplicativo.

Modulo	Entidades/Recursos	Telas/Fluxos	Status mobile atual
Autenticacao	User, JWT	login, signup-screen	Integrado com API (/auth/signup, /auth/login) e persistencia de token.
Feed	Truth, Dare	feed, feed-comments, create-challenge	Feed e create-challenge integrados; comentarios com estrutura pronta para backend.
Perfis	User	profile, settings	Perfil integrado em /users/me (GET/PUT) e logout funcional em

Modulo	Entidades/Recursos	Telas/Fluxos	Status mobile atual
			settings.
Interacoes	Like (truth/dare/club)	feed, cards de interacao	Like/deslike integrado com endpoints de truth, dare e club.
Clubes/Grupos	Club, ClubMember, ClubPrompt	clubs, create-group, search	Fluxos e UI prontos; integracao backend pendente para dados reais.

4. REQUISITOS TECNICOS DO PROJETO

Definicao da arquitetura e ferramentas utilizadas.

Linguagem	TypeScript (backend), TypeScript/TSX (mobile)
Framework / Plataforma	Express 5 (backend), React Native + Expo + Expo Router (mobile)
Deploy Backend (estado atual)	Build de producao: prisma generate && tsc -p tsconfig.build.json. Start de producao: node dist/src/server.js. Prisma migrations: prisma.config.ts com datasource.url em env('DIRECT_URL'). Runtime Prisma: adapter PostgreSQL usando process.env.DATABASE_URL. Servidor HTTP: app.listen com host 0.0.0.0 e porta por process.env.PORT. Infra alvo: API no Render e banco PostgreSQL no Supabase.
Variaveis de Ambiente Criticas	Backend (runtime): DATABASE_URL, JWT_SECRET, PORT. Backend (migrations): DIRECT_URL. Mobile: EXPO_PUBLIC_API_URL. Testes backend: TEST_DATABASE_URL (via scripts/run-prisma-test-command.ts).
Runbook de Deploy (Render)	1) Build: npm run build. 2) Migrations: npx prisma migrate deploy (com DIRECT_URL configurada). 3) Start: npm run start. 4) Validacao: checar endpoint de health funcional (pendente formalizacao no projeto).
Automacao de Entrega	Nao ha workflow de CI/CD versionado em .github/workflows no estado atual. Nao ha arquivo de infraestrutura declarativa de deploy (ex.: render.yaml) no repositório. Publicacao atual ocorre por configuracao manual do ambiente Render/Supabase.
Arquitetura Mobile (estado atual)	Navegacao por Stack em expo-router (app/_layout.tsx) com rotas para autenticacao, feed, busca, clubes, perfil, comentarios e configuracoes. Tema global por ThemeContext (system/light/dark). Camada de dados centralizada em mobile/services/api.ts com parse padrao de resposta e token em AsyncStorage.
Banco de Dados	PostgreSQL com Prisma ORM (Supabase em producao)
Controle de Versao	Git/GitHub
Servicos de Terceiros	Render (hospedagem da API backend) e Supabase (PostgreSQL gerenciado em producao).
Testes Automatizados (estado documentado)	Backend: 17 arquivos de teste em backend/tests. Mobile: 2 arquivos em mobile/__tests__. Mobile cobre login e cadastro (estado inicial, habilitacao de botao,

	chamada de API e tratamento de erro). Relatorios em docs/backend/tests e docs/mobile/tests com suites recentes em status PASS.
Escopo Mobile Mapeado	17 telas em app/, 84 componentes reutilizaveis, 10 hooks de estado/fluxo, 6 arquivos de constantes de tema e UI, 6 arquivos de tipos.

5. IDEIAS FUTURAS (BACKLOG)

Funcionalidades planejadas para versoes posteriores.

- Implementacao completa de fluxo de recuperacao de senha no backend e no app.
- Sistema de notificacoes push para novos desafios e interacoes.
- Melhoria da moderacao de conteudo e denuncias por usuario.
- Gamificacao com niveis, badges e ranking de participacao.
- Publicacao de evidencias e relatorios exportaveis em PDF dentro do app.
- Implementar CI/CD com pipeline de testes e deploy automatizado para Render.
- Versionar arquivo de infraestrutura de deploy (render.yaml) e checklist de rollback.

ANEXO A - EVIDENCIAS TECNICAS IMPLEMENTADAS

Backend - rotas principais

Base	Endpoints principais
/auth	POST /signup, POST /login
/users	GET /me, PUT /me, GET /
/truths	POST /, DELETE /:id
/dares	POST /, DELETE /:id
/feed	GET /
likes	POST /truths/:id/like, POST /dares/:id/like, POST /clubs/:id/like

Backend - evidencias de deploy em producao

Arquivo	Configuracao observada	Finalidade
backend/package.json	"build": "prisma generate && tsc -p tsconfig.build.json"	Gera cliente Prisma e compila artefato de producao.
backend/package.json	"start": "node dist/src/server.js" e "main": "dist/src/server.js"	Define entrypoint compilado correto para execucao em producao.
backend/tsconfig.build.json	Build dedicado incluindo src/**/*.ts e excluindo testes	Evita artefatos de teste no bundle de deploy.
backend/prisma.config.ts	datasource.url: env('DIRECT_URL')	Usa conexao direta para migrations no banco Supabase.

Arquivo	Configuracao observada	Finalidade
backend/src/lib/prisma.ts	PrismaPg com connectionString em process.env.DATABASE_URL	Conexao de runtime da API no Render ao PostgreSQL do Supabase.
backend/src/server.ts	app.listen(PORT, '0.0.0.0')	Permite binding externo exigido pelo ambiente de deploy do Render.
Ambiente de producao	Render + Supabase	Define arquitetura de nuvem atual do backend online.

Mobile - consumo atual de API e estado de integracao

Fluxo mobile	Funcoes/Endpoints consumidos	Status
Autenticacao	signup(), login(), saveToken(), getToken()	Integrado com backend
Feed	getFeed(), toggleLike(), deleteTruth(), deleteDare()	Integrado com backend (com fallback local em falhas)
Criar desafio	getUsers(), createTruth(), createDare()	Integrado com backend
Perfil	getMyProfile(), updateMyProfile()	Integrado com backend
Configuracoes	removeToken()	Integrado para logout
Busca, clubes, comentarios, criar grupo	Hooks locais (useSearchScreen, useClubsScreen, useFeedCommentsScreen, useCreateGroupScreen)	UI pronta, integracao backend pendente